Your Agentic AI System Is Not One Thing

The 7 layers every production agent needs and the layer most engineers skip entirely.



Rajesh Mane

Senior AI Engineer

An Agent Is Not a Smarter Chatbot

🤖 A standard LLM call returns one response. An agentic system plans, calls tools, evaluates results, and loops until a goal is satisfied or a guardrail stops it.

🔧 Every production agent requires at least 7 distinct layers: perception, reasoning, short and long term memory, tool calling, orchestration, guardrails, and observability.

💸 An MIT Sloan study found that 80 percent of agentic AI effort goes to data engineering, stakeholder alignment, and governance rather than model selection or prompt tuning.

⚡ A single unguarded tool call in an agent loop can trigger cascading actions. Prompt injection via tool outputs is the primary mechanism behind the top risk in the OWASP Agentic Top 10.

📜 **THE SCOPE MOST TEAMS UNDERESTIMATE**

Teams that treat an agentic system as a bigger prompt get to demo quickly and to production slowly. Architecture decisions made in week one determine operational cost in month six.

The 7 Layers of an Agentic System

🧠 PERCEPTION AND REASONING: THE COGNITIVE CORE

The input layer parses user intent from text, images, audio, or API events. The LLM reasoning engine interprets context, sets sub goals, and produces a plan. Extended thinking in Claude or chain of thought in GPT-5 significantly changes plan quality on multi hop tasks that require synthesizing across sources.

💾 MEMORY: SHORT TERM, LONG TERM, AND SEMANTIC

Short term memory tracks the active session state including tool results and prior turns. Long term memory uses vector stores or knowledge graphs for retrieval across sessions. Semantic memory captures relationships between concepts, enabling agents to reason across prior work rather than starting from zero on each invocation.

🔧 TOOLS, ORCHESTRATION, AND SAFETY

Tool calling via MCP gives agents standardized access to APIs, databases, and external services. The orchestration layer manages retries, timeouts, and parallel sub agent execution. Guardrails and observability form the safety layer: every tool call, reasoning trace, and output must be logged and validated before reaching users.

How the Reasoning Loop Actually Works

THE MECHANICS BEHIND THE LOOP

1

Perceive, plan, then act on tool results

The agent receives a goal, queries memory for relevant context, generates a plan with sub steps, calls the first tool, and feeds the result back into its context before deciding the next action. Each iteration narrows the gap between current state and goal state.

2

Reflection gates the next action

After each tool result, the agent evaluates whether the goal is met, whether the result is trustworthy, and whether a retry or different tool is needed. Skipping this check causes upstream errors to propagate silently through all subsequent reasoning steps.

3

Termination requires an explicit stopping condition

Without a defined stopping condition, agents loop until the token budget is exhausted. Every production agent needs a max step limit, a goal completion check, and a fallback to human escalation when neither of those conditions triggers first.

UNBOUNDED LOOPS ARE THE SILENT COST KILLER

Teams report tool call chains running 40 to 60 steps before hitting budget limits. Set max steps at 15 for most tasks and require explicit completion criteria before the agent starts.

Memory Is Infrastructure, Not an Afterthought

CHOOSE YOUR MEMORY LAYER BEFORE YOUR FIRST PROMPT

1. **In memory state: fast but ephemeral**
   Session state stored in process memory resets on every restart. It works for development and single instance testing. Any stateful production agent needs an external persistence layer chosen and wired before the first integration test.

2. **Vector stores power cross session retrieval**
   Long term memory via embeddings in a vector store lets agents recall facts, prior decisions, and user preferences across sessions. Retrieval quality depends on chunking strategy and embedding model alignment with the query distribution.

3. **Semantic graphs connect concepts across agents**
   Knowledge graphs capture relationships between entities rather than document similarity. Multi agent systems where agents share knowledge require graph memory to avoid redundant reasoning and contradictory conclusions across agent boundaries.

MEMORY ARCHITECTURE CANNOT BE RETROFITTED SAFELY

Teams that begin with in memory state and plan to migrate later spend 3 to 6 weeks refactoring state schemas. Memory management must be a first class infrastructure decision from the start.

MCP vs A2A: Two Protocols, Two Jobs

STANDARD PLUMBING BEFORE CUSTOM MIDDLEWARE

MCP and A2A solve different architectural problems and are designed to work together, not compete.

MCP: AGENT TO TOOL COMMUNICATION

MCP standardizes how agents call tools via JSON-RPC. All major frameworks support it natively. Use it for every API or service your agent needs.

A2A: AGENT TO AGENT COMMUNICATION

A2A lets agents on different frameworks invoke each other through Agent Cards. A LangGraph agent can delegate to a CrewAI specialist — no custom middleware needed.

MCP FOR TOOLS, A2A FOR AGENTS

Building custom middleware instead of adopting MCP and A2A creates vendor lock in and doubles future integration work. Frameworks without A2A support will become architecturally isolated as multi agent systems scale.

Why Demo Accuracy Lies About Production

DEMO ACCURACY VS PRODUCTION RELIABILITY

1

Happy path evals miss adversarial tool inputs

Demo evals use clean, cooperative inputs. Production agents receive malformed API responses, partial data, and prompt injected tool outputs. Testing only the happy path conceals the failure modes that matter most in real deployments.

2

Single agent accuracy does not predict multi agent accuracy

Each handoff between agents introduces a new point of context loss. In a chain of 4 agents each running at 95 percent accuracy, end to end accuracy drops to around 81 percent before any tool errors are counted.

3

Token budget evals omit latency and cost dimensions

An agent that answers correctly in 80 tool calls is not production ready. Evaluation must include cost per task, step count distribution, and latency at the 95th percentile, not just whether the final answer is correct.

BUILD ADVERSARIAL EVALS BEFORE LAUNCH

Include at least 20 percent adversarial cases in your eval set: malformed tool responses, contradictory memory states, and injected instructions embedded inside external data sources the agent retrieves.

LangGraph vs CrewAI vs OpenAI SDK

Dimension	LangGraph	CrewAI	OpenAI SDK
Best for	Stateful workflows	Role-based teams	OpenAI stack
Production ready	Highest (LangSmith)	Medium (growing)	High (built in)
MCP support	Via LangChain	Native (v1.10)	Native
A2A support	Via LangSmith	Native (v1.10)	Not yet
Memory depth	Explicit checkpts	4 of 5 types	Managed memory
Setup speed	Slower (60+ lines)	Fast (20 lines)	Fast (clean API)

💡 **START WITH CREWAI, HARDEN WITH LANGGRAPH**

CrewAI gets multi agent workflows running in under an hour. When you need durable execution, typed state, and production monitoring, migrate the critical paths to LangGraph incrementally.

When Multi Agent Earns Its Complexity

THREE SCENARIOS WHERE ORCHESTRATING AGENTS PAYS OFF

1. **Parallel specialist execution with interdependent outputs**
   Research workflows requiring simultaneous web search, database query, and code execution benefit from specialized sub agents running in parallel. A single agent serializes all three calls, tripling latency with no accuracy improvement over the parallel alternative.

2. **Long horizon tasks that exceed a single context window**
   Tasks spanning multiple documents, hours, or iterative refinement cycles require agents to hand off context explicitly. Hierarchical summarization between agents is more reliable than loading 200K tokens into one call and hoping for coherent synthesis.

3. **Cross functional tasks need domain specific grounding**

   A sales pipeline agent coordinating a CRM lookup agent, a pricing agent, and a drafting agent outperforms a single generalist because each specialist is prompted for its domain without crowding the others out of context.

💻 AGENT COUNT MULTIPLIES DEBUGGING COST

Each agent added to a workflow roughly doubles the surface area for failures. Measure single agent accuracy on your actual task before adding orchestration.

When a Single Agent Is the Right Answer

THREE CASES WHERE MULTI AGENT IS OVER ENGINEERING

1. **Tasks that complete in under 5 sequential tool calls**
   Document summarization, single source Q and A, and form extraction rarely benefit from a second agent. Adding orchestration to a linear task introduces retry logic and state management with no measurable accuracy improvement over a well prompted single agent.

2. **Latency sensitive flows needing sub 3 second responses**
   Every agent handoff adds 500 milliseconds to 2 seconds of overhead. Customer facing flows where users expect near instant replies cannot absorb multi agent coordination latency without measurably degrading the perceived experience.

3. **Tasks where a unified prompt outperforms role splitting**
   Code review, SQL generation, and classification perform better when the model receives the full problem in one prompt. Splitting across agents fragments context and introduces handoff errors that would not exist in a single well scoped call.

CLASSIFY YOUR TASK BEFORE CHOOSING YOUR TOPOLOGY

Ask: does this task require true parallel work, persistent memory across days, or domain specialized grounding? If the answer to all three is no, start with a single agent and a tool list.

Guardrails Are Not Optional in Production

BUILD SAFE BEFORE YOU BUILD FAST

1. **Validate every tool input and output, not just model output**
   Prompt injection arrives through tool results, not only user messages. A malicious web page returned by a search tool can hijack the agent's next action entirely. Input validation must cover every data source entering the reasoning loop.

2. **Apply progressive trust escalation for irreversible actions**
   Read actions can be autonomous. Write actions should require confidence thresholds. Delete or financial actions require human approval gates. This three tier model limits blast radius without blocking the agent on routine read only operations.

3. **Log every reasoning step as a first class audit event**
   Research shows 45 percent of executives cite lack of agent visibility as their primary adoption blocker. Structured traces at the span level covering each tool call, its inputs, and its outputs are mandatory before any regulated deployment.

OWASP AGENTIC TOP 10 IS YOUR MINIMUM BAR

OWASP released a dedicated Top 10 for Agentic Applications. Prompt injection, excessive agency, and insecure output handling are the top three risks.

Ship Your Agentic System in 4 Ordered Steps

RUN THESE STEPS BEFORE YOU SCALE

1

Define Task Scope and Topology

List every sub task and whether any require parallel execution or cross session memory. This drives whether you need a single agent, an orchestrator plus sub agents, or a persistent multi agent network via A2A.

2

Wire Memory and Tool Layer First

Implement the vector store, session state store, and MCP tool registry before writing orchestration logic. Teams that add memory later spend 3 to 6 weeks refactoring state schemas across the agent graph.

3

Build Evals Before Writing Prompts

Create at least 50 tasks including 10 adversarial cases before tuning any prompt. Evals written before prompts reveal capability gaps early. Evals written after prompts confirm what the developer already knew.

4

Add Guardrails at Layer Zero

Wire structured tracing, cost alerts, and output validation from the first deployment. Retrofitting observability into a live agent graph costs 3 to 5 times more than building it in from the start.



Rajesh Mane | Senior AI Engineer

Agentic AI Mistakes to Avoid

ERRORS ENGINEERS MAKE IN THE FIRST 90 DAYS

📌 Choosing a framework before defining the task topology: framework choice should follow task requirements. Starting with LangGraph for a task CrewAI handles in 20 lines adds weeks of boilerplate with no architectural benefit.

💸 Treating tool calls as free: every tool call in an agentic loop adds latency and cost. An unbounded agent running 60 tool calls can cost 30 times more per task than a well scoped 5 call equivalent on identical hardware.

📋 Skipping human in the loop for irreversible actions: agents that can delete records, send emails, or execute financial transactions without approval gates create liability the moment they encounter an edge case in production.

🔍 Evaluating only on correctness and ignoring step efficiency: an agent that takes 40 steps to answer a question a 5 step agent answers equally well is not production ready regardless of its accuracy score on the eval set.

📉 Deferring observability until after launch: 45 percent of enterprise leaders cite agent visibility as their primary blocker. Teams that ship without span level tracing cannot diagnose failures or demonstrate compliance to auditors.

A Production Agent Reality Check

> *An agentic system is judged not by what it can do in a demo but by what it refuses to do when the tool call chain breaks in production. Safety is not a constraint on capability. It is the capability that keeps everything else running.*

— Rajesh Mane, Senior AI Engineer

Key Takeaways

🧠

Architecture before prompts

Define task scope, memory topology, and tool registry before writing a single prompt. Teams that start with prompts spend weeks rebuilding infrastructure never designed for production scale.

💾

Memory is a deployment decision

In memory state. Dev only. Every production agent needs an external vector store and session persistence layer chosen before the first integration. Retrofitting costs 3 to 6 weeks.

🔄

Single agent first, always

Multi agent adds coordination overhead, debugging complexity, and latency. Prove a single agent with a focused tool list cannot handle the task before adding orchestration. Most tasks need only one.

🔬

Guardrails belong at layer zero

Tracing, output validation, and human approval for irreversible actions belong in the first deployment. The OWASP Agentic Top 10 is the minimum checklist before any agent touches production data.

🔔 **Follow Rajesh Mane for daily AI insights**

I share practical AI and ML content on topics like machine learning, deep learning, GenAI, Agentic AI, RAG, LLMs and MLOps daily.



👍 **Like** 💬 **Comment** 🔄 **Repost**